Direct cognition problem: a semantic skeleton of mathematics and its growth

Epochs limit active premises while certificates preserve proofs

Vasilii A. Gromov[1] and Nikolai N. Evdokimov[1*]

[1]Laboratory for Semantic Analysis, National Research University Higher School of Economics, 11 Pokrovsky Bulvar, Moscow, 109028, Russia.

*Corresponding author(s). E-mail(s): nevdokimov@hse.ru;
Contributing authors: stroller@rambler.ru;

Abstract

We study the direct cognition problem of formal knowledge: constructing a verifiable proof skeleton without a final target theorem under limited computational resources. We introduce a semantic skeleton of mathematics within a programme that distinguishes direct, inverse, and control cognition problems. Our continuous, generational generator separates a bounded active set, an archive of first proofs, and a library of closed fragments. The generator reversibly deactivates premises without deleting certificates. The selector measures how fragments shorten the expanded proof forest and accounts for definitions and overlaps. An exponential lower bound on the number of explicitly expanded provable instances, together with a separate unboundedness result for the schema graph, limits exhaustive storage but does not imply that finite slices evade computation. A frozen panel of 144 source trajectories tests neutral continuation under resource guards. At its primary endpoint, learned half-selection with deferred replay increases mean novelty over ungated growth by 95.8% and 30.3% at propositional sizes 25 and 200, respectively, with eight positive pairs out of eight in each regime. A locked comparison across 16 sources and 963 human targets finds no confirmed adaptive advantage. The fixed extended repertoire initially covers 29 targets and averages 1.625 new common-residual targets at 90 operational processor-seconds. Network comparisons reveal modular structure and show how proof-edge conventions alter graph profiles. These results establish a certified method that preserves proofs and supports further growth, not a universal accelerator or an optimal skeleton.

Keywords: formal knowledge, automated reasoning, semantic skeleton, proof hypergraph, theorem generation, Metamath

ORCID (Nikolai N. Evdokimov): https://orcid.org/0009-0003-3756-8281

1. Introduction

The graph of all formally correct mathematical statements differs from the graph of statements that humanity treats as theorems and lemmas [1] (see Section 7.3). We call the latter object a semantic skeleton of mathematics. This article studies it and develops methods to generate it. We extend this concept to any formalisable knowledge system and call the resulting structure a semantic skeleton of formal knowledge. The skeleton provides a compact active representation through anchor statements and links to expandable proofs; it neither depicts the complete theorem space nor automatically discovers a system of mathematical concepts. Here, 'semantic' means that the skeleton preserves formal statements, proof roles, and application conditions, whereas any substantive affinity with human mathematics requires a separate test. For knowledge-based reasoning, we ask which proved statements search should retain as premises under limited resources.

Our research programme distinguishes three cognition problems – direct, inverse, and control cognition problems; this paper addresses the first. The **direct cognition problem** generates a proof skeleton without a final target theorem. We aim not merely to generate correct mathematical statements[1], but to generate those that belong to the semantic skeleton. **The inverse problem** takes a particular statement and seeks intermediate lemmas that connect it to the existing base (current skeleton), to certify its status relative to the chosen formal system. This requires a proof of the statement, a proof of its negation, or a separate independence argument; unsuccessful search returns unknown. The **control problem** selects a growth regime to obtain a graph with desirable properties such as modularity, concentrated reuse, or a particular degree distribution. Tuning towards an external structural objective belongs to control, and success on a target theorem belongs to the inverse problem; neither supplies evidence that a topology emerges spontaneously.

The present paper addresses the direct problem by generating a semantic skeleton of mathematics under a fixed computational budget. We evaluate premise-pair selection with deferred replay and compare structural rule repertoires in neutral continuation tests. We measure new canonical formulae, resource costs, and additional coverage of a common set of human statements that no method initially covers.

The paper proposes (1) a formal contract, (2) an executable architecture, and (3) a reproducible comparison. The contract distinguishes proof status, activity, and

[1] Bachmair and Ganzinger [2], in *Resolution Theorem Proving*, describe saturation through inference and redundancy criteria. Wernhard [3], in *Generating Theorems by Generating Proof Structures*, studies target-free theorem generation. Section 2 distinguishes these foundations from the architecture and future-use test that we develop here.

certificate preservation, as well as schemata and their explicit instances[2]. The architecture supports continuous generational growth through a bounded active set, reversible deactivation, and an append-only certificate log; it separates the fragment library from decisions to activate fragment conclusions as premises. We separately test whether the method preserves proofs, shortens expanded representations, and supports productive subsequent search.

2. Related papers

The problem connects automated theory exploration, inference control, and proof reuse. Four distinctions guide comparisons: whether a method generates a statement together with its proof or first proposes a conjecture; whether it receives a final target; what it excludes from computation; and how it measures usefulness. These distinctions carry more weight than the mere presence of a graph, queue, or lemma library.

2.1 Saturation and search-resource limits

Bachmair and Ganzinger [2], in their handbook chapter *Resolution Theorem Proving*, explain how saturation combines inference rules with redundancy criteria. Schulz [4], in *E — A Brainiac Theorem Prover*, and Kovács and Voronkov [5], in *First-Order Theorem Proving and Vampire*, describe how clause indices, scores, and simplification direct that search. Target-free forward inference therefore supplies background rather than our novelty claim. We retain a long-lived archive of checked consequences and assess the skeleton through new growth under bounded activity.

Riazanov and Voronkov [6], in *Limited Resource Strategy in Resolution Theorem Proving*, adapt permissible clause weights to remaining time and search statistics. Their strategy directly precedes resource-based pruning. Logical redundancy and low expected computational return support different guarantees: a suitable redundancy criterion can preserve completeness, whereas a heuristic return estimate need not. Our age-based pruning protects earlier certificates but does not ensure fair future enumeration.

Hoder and Voronkov [7], in *Sine Qua Non for Large Theory Reasoning*, reduce the initial theory for a particular problem through SInE. Blanchette and colleagues [8], in *A Learning-Based Fact Selector for Isabelle/HOL*, learn fact relevance from earlier proof dependencies. Jakubův and Urban [9], in *ENIGMA: Efficient Learning-Based Inference Guiding Machine*, rank clauses during search according to how they contribute to successful proofs. These studies distinguish how a prover initially chooses premises from how it controls search. We additionally distinguish storing a fragment from admitting its conclusion to a bounded active slot, without labelling success against a final target.

[2]A formal contract states checkable guarantees (Definition 1). Proof status records whether a certificate passes the checker; activity permits use as a premise. A certificate records replayable proof steps. A schema contains expression variables; an instance substitutes admissible expressions consistently. Preserving a certificate retains its proof steps and dependencies.

2.2 Theory exploration and the usefulness of new statements

Colton [10], in his monograph *Automated Theory Formation in Pure Mathematics*, develops automated concept and conjecture discovery. Smallbone and colleagues [11], in *Quick Specifications for the Busy Programmer*, present QuickSpec, which enumerates terms, tests conjectural equalities, and reduces redundancy through their dependencies. Claessen and colleagues [12], in *Automating Inductive Proofs Using Theory Exploration*, combine such exploration with inductive proof in HipSpec. These systems show how auxiliary statements can support later work; our archive instead accepts each statement together with its checked derivation.

Johansson and colleagues [13], in *Hipster: Integrating Theory Exploration in a Proof Assistant*, validate conjectures through Isabelle/HOL and assess interestingness against ordinary and stronger proof mechanisms. Singher and Itzhaky [14], in *Theory Exploration Powered by Deductive Synthesis*, combine symbolic enumeration, rewriting, congruence closure, and inductive checking in TheSy. Their bounded solver also examines mutual derivability. These studies motivate separate measures of novelty, proof status, and usefulness; our archive checks every accepted derivation, whereas our experiments assess continued growth rather than mutual derivability of complete theories.

McCasland, Bundy, and Smith [15], in MATHsAiD: Automated mathematical theory exploration, describe theorem discovery from supplied axioms and definitions and distinguish it from proving a supplied goal. This distinction also frames our direct problem. We distinguish archived certificates, reusable fragments, and active premises in our experiments, and measure how much continued generation costs.

2.3 Generating proof structures: the closest predecessors

Wernhard and Bibel [16], in *Investigations into Proof Structures*, distinguish proof-tree size from minimal directed acyclic proof-graph size and study level-wise enumeration, theorem-size bounds, and bounded caches. Their analysis directly informs our generator. We extend the experimental focus to persistent certified growth and separately control archived proofs, reusable fragments, and active premises.

Wernhard [3], in *Generating Theorems by Generating Proof Structures*, combines target-free enumeration, inductive levels, lemma synthesis through shorter proof representations, and comparison with human Metamath theorems. We build on this precedent by growing certified knowledge continuously, reversibly controlling activity, and measuring the costs of bounded retention leases. Our comparisons separate fragment recurrence from additional output when search admits fragment conclusions.

Rawson and colleagues [17], in *Lemmas: Generation, Selection, Application*, separate those three stages and compare structural usefulness scores with how proving work changes after they add lemmas. Their tests motivate our distinction between shorter proof code and faster subsequent search. They assess lemmas through proof tasks; our principal trajectories instead continue without an external final target.

Wernhard and Zombori [18], in their preprint *Mathematical Knowledge Bases as Grammar-Compressed Proof Terms: Exploring Metamath Proof Structures*, encode libraries through compressed proof terms, including parametrised fragments. Our

code-length criterion scores particular closed first proofs, whereas a separate compiler derives general closed schemata from open modus-ponens programmes. The archive already shares subproofs. Consequently, shorter expanded forests measure the declared proof-term code rather than additional compression of the archive graph or universal value of lemmas for human mathematics.

2.4 Acquired knowledge: from recurrence to computational return

Minton [19], in *Quantitative Results Concerning the Utility of Explanation-Based Learning*, measures how applicability checks and maintenance can outweigh gains from acquired knowledge. In our setting, premises compete for a bounded number of active slots. Frequent fragment occurrences establish reuse, but only a costed continuation test can assess whether access to those conclusions yields more new results. We therefore report code-length savings separately from computational return.

Kaliszyk and Urban [20], in *Learning-Assisted Theorem Proving with Millions of Lemmas*, select intermediate lemmas through acquisition cost, reuse, centrality, and greedy changes to the dependency graph. They also distinguish protocols by access to future proofs. Our selector recalculates overlaps using only the preceding forest, because earlier choices change the value of another fragment. Development diagnostics measure human-statement coverage after a run; a separate locked comparison evaluates a previously unscored target pool.

2.5 Trainable generators and growing libraries

Wang and colleagues [21], in *LEGO-Prover: Neural Theorem Proving with Growing Libraries*, build and reuse verified auxiliary skills while solving formal tasks. Yang and colleagues [22], in *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, combine premise retrieval with search and track fact availability. These systems inform future uses of a skeleton for target-driven reasoning. Our historical direct-growth panels start from axioms, whereas the native extensions inherit a certified source library. Neither calls a language model for formulae or fragments.

Poesia and colleagues [23], in *Learning Formal Mathematics from Intrinsic Motivation*, jointly learn conjecturing and proving in MINIMO from an initial axiomatisation. Ota and colleagues [24], in their preprint *Self-Supervised Theorem Discovery in a Formal Axiomatic System*, turn discovered theorems into prover actions and select them through generalisation and difficulty of rediscovery. Both systems create their own search targets. Our generator instead continues forward without a trainable target-setting loop.

Lin and colleagues [25], in *ATG: Benchmarking Automated Theorem Generation for Generative Language Models*, assess generated theorems partly through their usefulness for subsequent proving. We test neutral direct continuation and additional coverage of human statements. This choice measures further target-free growth; independent goal-directed tasks would test the complementary inverse problem.

2.6 Prover cooperation and graphs of mathematical libraries

Tomaszuk and colleagues [26], in *MMLKG: Knowledge Graph for Mathematical Definitions, Statements and Proofs*, represent the existing Mizar library as a knowledge graph. Li and colleagues [27], in their preprint *The Network Structure of Mathlib*, distinguish dependency levels and relate observed centrality to formalisation infrastructure. These studies motivate a graph-level account of mathematical libraries. Our generator additionally requires joint premises and expandable certificates; high degree, short paths, or a compact projection cannot replace those proof conditions.

2.7 Position of this work

Our method assigns each proven formula three distinct roles: an archived result, a reusable library entry, and a possible active premise. Certificate replay checks the first role, past savings in proof code help select reusable entries, and neutral continuation measures future use. Structural synthesis adds another route to library entries through checked deduction. We combine these mechanisms with explicit access permissions and test their incremental effects against fixed controls.

3 Formal model and outcome criteria

3.1 Calculus and proof state

A proof certificate records elementary inferences and substitutions that a checker can replay from the initial base. We call a statement proven once its certificate passes those checks, and active when the generator may use it as a premise. A schema contains variables that stand for admissible expressions; an instance replaces those variables consistently. A configuration fixes the language, proof rules, and contextual requirements:

$$\mathcal{C} = (\Sigma, \Gamma, \Re, \nu, \Xi),$$

where Σ specifies the language, Γ denotes the initial axioms or schemata, $\Re$ contains the rules, ν maps each statement to its declared canonical form, and Ξ describes contextual restrictions. In systems with quantifiers and hypotheses, each vertex includes the necessary context, variable types, and admissible-substitution conditions alongside its formula. Formula identity alone therefore need not identify the full proof state.

A typed hypergraph[3] $H = (V, E)$ provides the primary representation. We record a rule application as

$$e = (p_1, \ldots, p_k; \rho; \sigma; \chi; q),$$

where p_i denotes premises in role order, ρ denotes the rule, σ denotes the substitution, χ contains side conditions, and q denotes the conclusion. Each hyperedge requires

[3] A typed directed hypergraph records statements as vertices, together with their variable types, hypotheses, and side conditions. A directed hyperedge joins an ordered group of premises to one conclusion and records the rule and substitution. Its premise order identifies rule roles; the inference requires those premises jointly. For modus ponens (MP), the minor premise p and major premise $p \to q$ jointly yield q; neither premise alone supplies that inference.

all its premises jointly, whereas alternative hyperedges represent different derivations of the same conclusion. The representation thus preserves the distinction between jointly necessary premises and alternative proofs.

To canonicalise means to remove only the declared redundancy of notation. In the implicational fragment, the procedure normalises variable names by first occurrence without identifying arbitrary logically equivalent formulae. A canonical identifier therefore does not turn a vertex into a semantic equivalence class.

3.2 Skeleton contract

Definition 1. A certified proof skeleton consists of a triple

$$\mathcal{S} = (S, \Omega, \mathsf{Expand}),$$

with the following properties. A finite certificate in Ω supports every anchor vertex $v \in S$; certificate dependencies follow a well-founded order; Expand expands an anchor-lemma reference into a proof from the permitted initial base; and every elementary step passes rule and side-condition checks. Together, these requirements let a checker recover each anchor independently from that base.

The contract does not require every known alternative proof. An implementation may preserve one correct witness, several variants with different costs, or a common structure that shares subproofs. Deleting the sole expandable witness nevertheless violates the contract unless another certified reconstruction remains available.

'Semantic' carries a restricted operational meaning here. The skeleton preserves formal statements, their proof roles, and application conditions, whereas we must test separately whether the skeleton resembles how humans organise mathematics. In particular, provable equivalence among all theorems relative to a common theory provides too coarse a relation for identifying useful anchor lemmas.

3.3 Reconstruction and new growth

Proof expansion and new growth serve distinct purposes:

$$\mathsf{Expand}(\mathcal{S}, v) \to \pi_v, \qquad \mathsf{Grow}(\mathcal{S}, b) \to \Omega_b.$$

The first operator returns a certificate for an existing anchor. The second performs forward inference under budget b without a final target formula, using the initial axioms, selected anchors, and its own new consequences. Vertices that serve only as proof support do not automatically enter the available premise set, so archival presence alone does not confer search activity.

New-growth counts exclude rediscovery of formulae that the archive already contains. Let Ω_0 denote the common initial archive from which the comparison policies choose anchors, and define the main direct-continuation measure as

$$N_b(\mathcal{S} \mid \Omega_0) = |V(\Omega_b) \setminus V(\Omega_0)|\,.$$

This measure counts syntactic novelty under the declared canonicalisation, even after earlier formulae leave the active skeleton. It does not establish mathematical significance for every new result.

The later native tests use typed one-way coverage with mandatory distinct-variable conditions. For source block j, let T denote the external reference and let $C_T(S_{j,m}, 0)$ denote the statements that method m covers before further search. We freeze a common residual

$$R_j = T \setminus \bigcup_m C_T(S_{j,m}, 0), \qquad Q_j(m, h) = \left| R_j \cap C_T(S_{j,m}, h) \right|.$$

The observer measures coverage of committed results; it supplies no premise, reward, or proposed conclusion. The residual removes every method's initial coverage advantage within a block. Counts of Q_j test new reference capability, whereas N_b counts syntactically new formulae outside the common archive. Neither measure substitutes for the other.

Resource accounting distinguishes the active set and reserve from the library, proof support, and archive. We measure their sizes separately, together with peak memory for the entire process; a fixed premise count does not imply fixed total memory. The principal size bounds $L = 25,200$ count every syntax-tree vertex, not only leaves, and therefore constrain a different quantity from the active-set limit.

3.4 Useful skeletons and empirical Pareto sets

A useful skeleton preserves checked proofs and supports further inference within the declared budget. The proof requirements guarantee the first property; neutral continuation tests the second. This operational criterion links proof preservation to the work that the retained library supports.

For each policy and anchor cap, we record a triple: the measured cost of constructing and using a skeleton, the bytes in its proof-backed package, and the new schemata that it generates during continuation. One setting dominates another only when it uses no more CPU or package bytes, yields at least as many new schemata, and improves at least one coordinate. The empirical Pareto set contains the nondominated settings within the declared finite comparison grid. This definition avoids assigning an arbitrary exchange rate between a byte, a CPU-second, and a new formula.

We compare settings only within one logic–size regime, one source ceiling, one continuation horizon, and a common exclusion archive. Package bytes include both anchors and certificate support; an anchor count alone omits that support.

4 Limits of exhaustive explicit construction

The combinatorial argument depends on the unit of storage. Let T_L contain the syntactically distinct formulae over one fixed letter p that admit proofs in which every formula, including axiom instances, contains at most L syntax-tree vertices. Size counts syntax-tree vertices, and the calculus contains the schemata $K : A \to (B \to A)$ and $S : [A \to (B \to C)] \to [(A \to B) \to (A \to C)]$, together with modus ponens. The resulting bound concerns explicit formula instances under these conventions.

Theorem 1. For every integer $m \geq 0$,

$$|T_{18m+17}| \geq Cat_m, \qquad Cat_m = \frac{1}{m+1}\binom{2m}{m} \sim \frac{4^m}{\sqrt{\pi}\, m^{3/2}}.$$

These conclusions contain $4m+3$ syntax-tree vertices and admit proofs with at most two applications of modus ponens along any path from an axiom instance. Appendix A presents the full proof and intermediate size calculations, using standard Catalan enumeration [28].

An exhaustive store that retains each such formula separately requires exponentially many records in L. No fixed total capacity can accommodate that representation for every increasing L. This output-volume bound does not supply a new lower bound on the complexity of finding a single proof, and it permits streaming output, growing storage, and exhaustive computation of any fixed finite slice.

The generator needs a separate schema-level bound. Every $F \to F$ instantiates the single schema $A \to A$, so the explicit family behind Theorem 1 collapses under schema storage. Its exponential bound therefore **does not transfer** to the number of canonical schemata in the archive.

Theorem 2. Let H_d^{sch} contain K and every most general result of schematic modus ponens up to depth d, with separate renaming of premise variables and canonicalisation only up to renaming. With no size or width limits, subsumption, or pruning,

$$|V(H_d^{sch})| \geq d + 1.$$

Appendix A proves the claim. Together, the bounds establish why an ever-growing exact graph eventually exceeds fixed retained capacity, distinguishing explicit instances from canonical schemata. They motivate a design that limits active work and preserves proof certificates separately. Appendix A details the assumptions, pair counts, and finite restrictions.

5 Continuous skeleton and proof library

5.1 Continuous growth with distinct states

The continuous variant keeps one generator across epochs and reserves no compulsory time share for a second branch. We track its state as

$$X_e = (A_e, P_e, D_e, \Omega_e, \mathcal{I}_e, B_e),$$

where A_e contains active premises, P_e holds the passive queue of proven candidates, Ω_e contains the certificate archive, $\mathcal{I}_e$ contains working indices, B_e denotes remaining resources, and D_e contains root identifiers for closed library fragments whose certificates reside in Ω_e. The program defines retained anchors as $S_e = A_e \cup D_e$, without an additional independent state component. The two sets serve distinct purposes: A_e supplies search premises, whereas D_e stores references to multistep proofs that the checker verified. Their ancestor proof support $Anc(A_e \cup D_e)$ serves checking and expansion, not an additional active base.

The archive grows through new canonical statements and their first proofs. The policy permanently protects only the initial axioms in the working set. The policy

retains other active formulae for finite periods and extends those periods when a formula contributes to the first proof of a new result; library-selected formulae receive a separate finite period. Deactivation removes a formula from search indices but preserves its archive entry. If search rediscovers that formula, the policy can return it to the passive queue after a specified delay and within a return quota, measuring its new age from queue re-entry rather than old proof depth. These operations neither change the certificate nor turn a derived lemma into an axiom.

Explicit limits constrain active derived formulae and passive candidates after every layer. The policy first preserves the new search frontier, then accounts for finite library-retention periods and recent productive use. The policy sets per-layer selection width separately from total active-set size. Since the archive retains every entry, bounded activity does not imply constant total memory.

5.2 Utility of a reusable proof fragment

A closed fragment represents a particular first proof of a lemma with initial axioms at its leaves. A closed fragment excludes unproven open premises but may contain schema variables. To expand it, the checker follows each internal vertex's archived premises in their original order. This coding stage names existing proofs. The separate structural compiler below generalises short open proof programmes and derives their closed implication schemata. Both routes retain elementary certificates and leave the original consequence relation unchanged.

Fragment cost accounts for repeated premise positions. Let R_e denote a finite set of recent proofs for selection, and let $pa(v)$ denote the ordered parent list of vertex v, allowing repeated entries. Counting each premise occurrence with its multiplicity gives the expanded proof-term size

$$t(v) = 1 + \sum_{p \in pa(v)} t(p).$$

The unit counts an axiom name, a rule application, or a fragment reference. For a chosen library D, define

$$c_D(v) = \begin{cases} 1, & v \in D, \\ 1 + \sum_{p \in pa(v)} c_D(p), & v \notin D, \end{cases}$$

$$d_D(v) = 2 + \sum_{p \in pa(v)} c_D(p).$$

The quantity $d_D(v)$ includes a definition header and its root instruction, whereas references to other definitions must follow the original well-founded dependencies. Thus the encoding gain for the proof forest accounts for both references and definitions:

$$G(D; R_e) = \sum_{v \in R_e} t(v) - \left[\sum_{v \in R_e} c_D(v) + \sum_{v \in D} d_D(v) \right].$$

The selector admits fragments only while they reduce code length; it need not fill its quota. Candidates comprise multistep proofs of height at least two that occur at

least twice in the expanded forest; a bounded pool ranks them by how much each could reduce repetition. The greedy selector then adds the fragment with the largest positive increment in G and recalculates overlaps. The selector stops when no positive increment remains, so the procedure searches only a bounded candidate pool and does not guarantee optimal compression.

Lemma 1 (library expansion). For a finite acyclic archive, this encoding expands into the original forest of first proofs. If the selector starts from an empty library and admits only strictly positive increments in G, its final gain on the training forest remains non-negative. Appendix B.5 proves the lemma.

The lemma guarantees that the codec recovers the original proofs without increasing code length on the selection forest. The gain G measures savings in expanded proof-term symbols; the archive already shares identical subproofs. The continuation experiments separately measure whether library conclusions improve future search under a CPU budget.

5.3 Continuous construction cycle

A generation denotes a layer of new consequences, whereas an epoch marks when the program updates search state. Epoch number does not equal minimum proof height from the initial axioms. Unlike earlier restarts from selected anchors, this algorithm preserves one generator and its unfinished work; the experimental protocol specifies the retention periods, quotas, and bounds that govern that continuity.

Algorithm 1. Continuous certified growth.

1. Check the initial axioms or inherited certified library, create the working indices, and initialise the archive and reusable dictionary.
2. While resources permit another layer, select an admissible frontier from the queue according to the fixed policy and its tie-breaking rule.
3. Process available rule applications. Check substitutions and side conditions, and canonicalise conclusions. Store a first proof for each new formula; handle rediscovery separately without increasing the new-formula count.
4. Update productive-use records and finite retention periods. Apply active-set and queue bounds while preserving certificates for inactive formulae. Return eligible rediscoveries to the queue subject to the specified delay and quota.
5. At the designated epochs, select a library from the preceding proof forest. Apply bounded retention to selected conclusions within ordinary premise slots while preserving their archived certificates.
6. Store the new certificate increment and state snapshot. Credit only a fully completed layer within budget, and record the stopping reason.

At a library-review point, the structural extension may add checked lemma schemata before the upper codec runs. It preserves the preceding interface and charges new definitions; it does not replace the accumulated generator with a new search from a hand-picked theorem list.

Epochs limit active premises while certificates preserve proofs.

The algorithmic budget includes fragment selection, index updates, internal checking, and storage. The independent checker replays the complete log after the experiment;

we account for its cost separately. The pseudocode consequently specifies dependency order among operations, not equal execution times for successive layers.

5.4 Soundness and the limit of the guarantee

Every archive certificate expands to the initial base provided the axioms satisfy the configuration, each new step passes checking, and each reference points to an earlier certified vertex. Appendices B.1–B.2 state the formal guarantees and prove them. These guarantees preserve existing proofs when the policy deactivates premises, but do not ensure complete enumeration: an inactive premise can prevent a useful future rule application.

Adding a provable lemma does not strengthen the underlying logic, but can change which proofs the available resources can reach [29]. Under size and height bounds, we must account explicitly for intermediate formulae when we expand proofs; Appendix B and Section C.1 provide the full resource conditions and a specific identity example. Complete finite saturation additionally requires the process to execute every available rule application eventually; the present bounded policy does not guarantee that schedule.

5.5 Learned pair selection with deferred replay

The later variant controls pairs of candidate premises before it unifies them. A frozen model scores only premise syntax and existing first-proof heights; it never inspects a proposed conclusion. Its training labels mark first-proof ancestry that supports an earlier compression dictionary. The policy retains roughly half of compatible candidates, includes a fixed exploratory share, and preserves their production traversal order. Thus it predicts a corridor towards useful proof support rather than the truth of an unseen theorem.

Deferred replay retains some skipped pairs in a bounded queue. Each epoch can test up to 128 ready pairs and reserve up to four ordinary premise slots to restore missing parents. A replay uses only currently active, certified premises; it does not introduce a new axiom or activate cold proof support. Queue capacity, overflow, and the cost of scores remain explicit. This mechanism delays the point at which the hard filter saturates while preserving the same elementary proof checker.

The policy constructs an archive, then extracts proof-backed anchor packages for neutral continuation. The reported chain CPU includes work to build the source, choose packages, and continue search. Training and the later independent audit enter separate accounts. Code-length gain chooses candidate support, and continued growth measures operational usefulness. These separate tests allow a compact package to succeed without claiming that every retained formula belongs to a unique mathematical core.

5.6 Additional compression above the accumulated library

The upper codec preserves a base dictionary D_0 and adds a definition only when its joint marginal gain remains positive. It recalculates nested references and definition costs whenever it adds a definition and imposes no target dictionary size. For an eligible fragment, let r count its expanded occurrences in the current forest and dictionary

bodies, and let b count the symbols in its current body. The exact saving under this code equals

$$\Delta(v \mid D) = (r-1)(b-1) - 2.$$

The two charged symbols account for a new definition's root and header after the reference savings. Appendix F.1 proves this identity and its telescoping guarantee. Positive increments improve the previous encoding, even when D_0 already contains mandatory entries; they do not guarantee a globally shortest code, fewer archive bytes, or faster future inference.

5.7 Structural synthesis of general lemma schemata

The structural layer abstracts a short MP proof tree, replaces its boundary by hypotheses, and considers possible equalities among those hypotheses. Joint typed unification solves all MP constraints together. For each consistent programme and order of its distinct boundary hypotheses, the compiler applies the deduction theorem and derives a closed implication schema. The generalisation rule (Gen) cuts an open fragment: the compiler never moves a deduction step through unrestricted Gen.

For example, a programme with boundary p, $p \to q$, and $q \to r$ concludes r. An appropriate discharge order yields

$$(p \to q) \to ((q \to r) \to (p \to r)).$$

The compiler recovers this familiar composition law without a human theorem target. It expands every new step through ax-1, ax-2, and ax-mp, or invokes an existing checked schema through an explicit admissible instance. All output certificates pass the native checker and the independent Metamath verifier. Proposition F.2 states the proof obligation; this compiler implements the standard deduction construction rather than a new deduction theorem.

The catalogue distinguishes a requested principal formula from the actual schema that its certificate supplies. A more general stored schema may cover several requests, so we deduplicate delivered statements and their applicability frames. This distinction explains why the number of candidates need not match the number of rules that the compiler delivers. The finite grammar and size guards bound the actual construction, not all possible useful lemmas.

5.8 Fixed structural repertoire

The final native architecture combines the cumulative constructor and positive-marginal codec with a finite structural repertoire. A common grammar delivers 217 checked schemata through three open MP nodes, including 16 short outputs through two nodes. The fixed quota-matched variant adds 12 extended outputs that the fixed ranking selects using pooled earlier development profiles. It uses no held-out theorem to select those rules. This fixed repertoire supplies a reproducible working configuration, not an optimum over all possible libraries.

The locked comparison separately tests whether the source history should change those 12 outputs, versus fixed, random, and whole-catalogue alternatives. The adaptive score belongs to that comparative experiment rather than the recommended architecture. Appendix F.5 specifies it fully, and the results retain its registered outcome.

5.9 Search permissions after the library freezes

The neutral test receives only declared interface statements as premises. It cannot use certificate-only support for sampling, subsumption, or rejecting a rediscovery as already known. If search derives a hidden support statement again, the checker first validates a new witness from currently permitted parents; only then does the statement gain search rights. A separate log retains that witness even when the old archive already stores the same statement.

The neutral continuers reset learned scores, deferred queues, and reuse counters. They apply either uniform premise sampling or the same fixed mixture of uniform, bridging, contracting, and frontier sampling to every arm. They retain admitted future results without the constructor's rot-pruning or active cap. Thus this test assesses a library's use under a common engine, not the effect of running each builder's controller longer. The mixed engine favours frontier statements according to their identifiers, so the experiment concerns delivered representations rather than an invariant semantic functional.

6 Materials and methods

6.1 Formal systems and the scope of the tests

The historical propositional panels use $\to, \neg$ with three axiom schemata:

$$ax_1 : A \to (B \to A),$$
$$ax_2 : [A \to (B \to C)] \to [(A \to B) \to (A \to C)],$$
$$ax_3 : (\neg A \to \neg B) \to (B \to A),$$

Schematic modus ponens renames premise variables apart before it unifies them. The modal condition adds the normal K axiom $\Box(A \to B) \to (\Box A \to \Box B)$ and necessitation for theorems. These two configurations differ in language, axioms, and rules together, so their contrast does not isolate a causal effect of logic alone.

The size bound $L \in \{25, 200\}$ counts abstract-syntax-tree nodes in each canonical formula that the generator stores.

An independent checker traces each new result to the original axioms. It implements its own canonical forms, occurs-check unification, and rule checks without importing the generator's corresponding code. The audit validates one first proof per vertex, not the complete set of alternative proofs.

Each random initial scheduler state defines one trajectory and hence one repeat. In the continuation experiments, a common source archive defines a block within which we pair the ways of choosing anchors. Thousands of formulae along one trajectory do not supply thousands of independent observations.

6.2 Native development and the one-shot holdout

The native tests inherit a certified 2,708-statement library on the explicit basis that the network study specifies. They continue the cumulative constructor rather than rebuild that source from axioms. The constructor retains its learned half-filter on supported implication/negation terms, intrinsic fallback scores on other native syntax, deferred opportunities, four ordinary fragment slots, and a cap of 128 active derived premises. Rot-pruning deactivates an unleased, never-used premise after eight idle blocks; cap enforcement can deactivate other premises as well. Neither operation deletes its certificate.

Our earlier development reference contains 919 unique compatible closed native statements; axiom instances already cover 34. This reference informed method development. We use it to identify overlap with the held-out corpus.

For the one-shot test, Wernhard's POI supplies an external benchmark of 1,374 implication-form propositional theorems from Metamath [3]. We exclude 402 alpha-exact matches to the preceding reference after unfolding the specified conjunction, disjunction, and equivalence definitions. Nine files retain named truth or falsehood constants outside the declared implication/negation language; we exclude rather than reinterpret them. The resulting corpus contains 963 unique targets. This filtering removes specified syntactic overlap, not all logical equivalence or one-way subsumption, and both references share Metamath ancestry.

The protocol fixes 16 new states of the random number generator (RNG) for source construction, five arms, and two neutral continuers, giving 160 trajectories but only 16 paired statistical blocks. Each common source runs 1,500 further opportunities. Arms receive the common compressed source and short catalogue, then add no extended rule, a fixed set of 12, a random set of 12, a source-adaptive set of 12, or all 201 extended entries. The full-catalogue control tests whether adaptation improves on supplying the entire finite structural repertoire rather than merely more rules.

The protocol requires all blind construction and continuation to finish and commit file hashes before the evaluator opens the targets once. No human target enters generation, features, admission, or the reward. The primary endpoint averages $Q_j(m, 60)$ across the two continuers within each source block. A second endpoint imposes 90 CPU-seconds of measured cold operational cost, charging new source work, the whole catalogue, arm preparation, and continuation setup before growth. Historical costs before the inherited checkpoint, interpreter startup, offline goal observation, terminal bulk export, and independent audit remain separate; this endpoint does not measure a complete from-axioms prover job.

We compare adaptive selection with fixed, random, and full-catalogue controls through exact one-sided paired sign flips, assuming sign exchangeability under the null. For each comparison, we retain the larger p-value from the two endpoints, then apply Holm's correction across the three controls. Confirmation requires an adjusted p below 0.05 and a mean gain of at least one residual target at both endpoints against every control. All planned cells and CPU horizons must finish, the selector must vary across sources, and the independent certificate and permission audit must pass. Missing cells, early guards, or a constant selector preclude confirmation. Appendix G records the frozen commitment and outcome.

The native bound of 160 constrains stored canonical formulae, and the certificate-height bound reaches 256. These bounds do not constrain every fully instantiated intermediate formula after expanding all substitutions, unlike the explicit-instance convention in Theorem 1. Native trajectories and the earlier propositional/modal panels therefore retain separate labels, metrics, and cost accounts.

7 Results

7.1 Neutral continuation and empirical Pareto trade-offs

The later frozen panel comprises 144 source trajectories, six pair-selection policies, three logic–size regimes, and eight scheduler states per regime. Each source supplies packages with up to 4, 16, or 64 anchors at nominal source ceilings of 60, 180, and 600 CPU-seconds. Separate neutral generators continue from those packages, giving 1,296 continuations with nested readings at 10, 30, and 60 CPU-seconds. The primary endpoint fixes the source ceiling at 180 CPU-seconds, the anchor cap at 16, and continuation at 30 CPU-seconds. Novelty excludes the union of concurrent source archives and the frozen training archives; Appendix D records the protocol and checks.

At this endpoint, the learned half-filter with deferred replay improves mean novelty over ungated growth in both propositional regimes, with eight positive pairs out of eight. The size-25 regime also improves the measured CPU and package-byte coordinates in every pair. The size-200 package grows by 16.5% on average, illustrating why a useful skeleton need not minimise every resource coordinate. The modal comparison splits four wins and four losses despite a positive mean (Table 1).

Table 1 Neutral-continuation outcomes for ungated growth and learned half-selection with replay

Regime	New schemata without gate	Learned half with replay	Positive pairs	Chain CPU without gate / with replay
Propositional, size 25	13,935.38	27,285.12	8/8	209.74 / 169.53
Propositional, size 200	20,569.25	26,805.12	8/8	147.54 / 136.06
Modal K, size 25	22,164.25	25,456.50	4/8	209.68 / 139.00

The table reports means across eight scheduler states, not independent mathematical theories. Chain CPU includes credited work to build sources, select packages, and continue search, whereas training and later independent audit remain separate. Resource guards stop 119 sources at the memory threshold, 21 through the implementation's saturation condition, and four at the epoch cap; no source completes 600 CPU-seconds. The gains therefore characterise the observed resource-bounded trajectories rather than a completed long-horizon limit or a fixed-quality speedup.

The Pareto comparison fixes the primary source ceiling and continuation horizon while varying six policies and three anchor caps. Each regime contributes 18 mean triples. A setting enters the three-coordinate frontier when no other setting uses no

more CPU or package bytes, yields at least as much novelty, and strictly improves at least one coordinate. The figure projects those triples onto CPU and novelty; outlined points mark the full three-coordinate frontier, so a point can remain nondominated through smaller package size even when its projection looks inferior (Fig. 1).

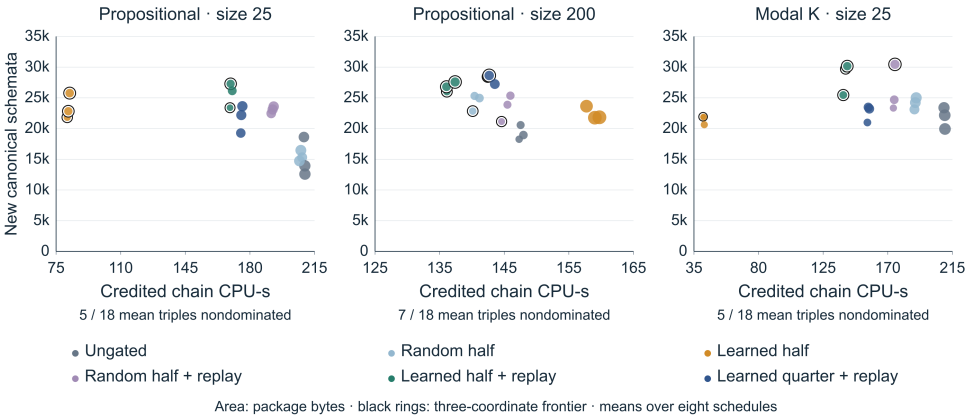


Fig. 1 Empirical Pareto trade-offs at a source ceiling of 180 CPU-seconds and continuation of 30 CPU-seconds. Colour identifies the pair policy, marker area indicates proof-backed package bytes, and outlines identify nondominated mean triples across CPU, bytes, and novelty. Each point averages eight scheduler states

These trade-offs favour different settings for different budgets. The learned half-filter without replay can cost less than replay at size 25, whereas replay yields more new schemata. More anchors or a later source slice need not improve continuation. The empirical frontier consequently supports a choice among useful finite skeletons, rather than a universal winner or an optimum beyond this grid.

7.2 Locked native comparison

All 16 source blocks and 160 planned continuations finish both registered horizons and pass independent replay. For each table entry, we first average the two neutral continuers within each source, then average across the 16 sources. Initial coverage counts all 963 reference targets; both future columns count only the common initially uncovered residual within that source block. Derived-anchor counts exclude 29 basis roots and separate certificate-only support (Table 2).

Table 2 Initial coverage, further common-residual coverage, and anchors in the locked native comparison

Library	Initial coverage	New at 60 growth s	New at 90 operational s	Derived anchors
Common core and short rules	25.00	1.312	1.344	633.6
Fixed extended repertoire	29.00	1.594	1.625	675.0
Random extended repertoire	26.38	1.500	1.562	680.4
Adaptive test variant	27.88	1.656	1.688	673.4
Whole fixed catalogue	44.00	1.562	1.594	1282.3

The whole fixed catalogue initially covers 44 targets, versus 29 for the fixed extended repertoire, but reaches 1.56 versus 1.59 new common-residual targets at 60 growth CPU-seconds. This contrast illustrates why initial coverage cannot rank future capability. The adaptive score chooses three distinct rule sets and reaches 1.656 new residual targets, yet its paired advantage over the quota-matched fixed repertoire averages only 0.0625 targets at either endpoint (Table 3).

Table 3 Paired adaptive-minus-control differences and Holm-adjusted tests at both registered horizons

Adaptive minus control	Difference at 60 growth s	Difference at 90 operational s	Holm adjusted p
fixed	+0.06250	+0.06250	1.000
random	+0.15625	+0.12500	1.000
fixed all	+0.09375	+0.09375	1.000

Every Holm-adjusted p-value equals 1.000, and no contrast reaches the registered mean margin of one additional target at both clocks. We therefore retain fixed structural rules in the working architecture and do not attribute a confirmed gain to adaptive catalogue ranking. Appendix G reports the full paired comparison, statistical rule, and technical amendment.

The independent audit reconstructs 4,174,478 admission events and 2,846 witnesses that record how search rediscovers hidden support, checks all target positions, and reproduces the endpoints and decision. These counts include repeated events across trajectories, not independent theorems. The comparison charges the full recorded catalogue to every arm under the shared backend; it does not test a specialised static implementation that avoids compiling unused entries.

7.3 Metamath and generated proof networks

The network comparison follows the cumulative method through its delivered library and certificate support. Panel b uses the first source, 2026102201, with the quota-matched fixed structural repertoire. Appendix H presents the profiles of both variants and the separately preselected adaptive image. Both representatives follow the first-source convention, independently of target coverage, graph appearance, and power-law p-values.

All objects share the explicit native basis from the pinned `set.mm` revision [1]: 25 logical or set-theoretic axioms and rules, six definitions, and 12 syntax assertions. Twenty-nine closed source schemata initialise search. The inherited proof-closed core contains 2,708 statements; the cumulative constructor continues that core, filters and revisits opportunities, applies reversible rot-pruning, compiles structural rules, and codes recurrent proofs. Panel b shows the delivered interface and every ancestor that its certificates require, not the full inherited archive. Appendix H records the new sources, projections, and metrics.

The first-source fixed library provides 675 derived anchors and 29 basis roots; 137 further vertices supply certificate-only support. Its projection therefore contains 841 vertices and 1,573 directed arcs. Across all 16 fixed sources, anchor counts range from 665 to 682, with a mean of 675.0; projected sizes range from 836 to 852. Hidden support does not confer search rights, and a verifier can expand every anchor to the original basis.

An arc runs from a premise to the conclusion that cites it. We omit primitive rule vertices and collapse repeated arcs in every object. Certificates retain joint premise roles, substitutions, and side conditions that this ordinary graph projection cannot express. The human fragment contains 2,306 named assertions whose actual proofs use only the declared basis, including assertions with essential hypotheses. Native vertices denote closed schemata with mandatory distinct-variable conditions. A shared basis does not eliminate the difference between named human steps and elementary native inferences.

The full reference enumerates all admitted MP steps through most-general unification and prescribed generalisations through proof height three, without skeleton-selection or formula-size cuts. Its 11,772 statements support both first-proof and all-inference projections. This reference supplies a complete finite slice of the stated operator, not all substitution instances, a deductive fixed point, or infinite closure of Zermelo–Fraenkel set theory with choice (ZFC). Panel b reaches certificate height 23; the comparison holds neither depth nor construction cost constant (Fig. 2).

The representative skeleton reaches modularity 0.622, versus 0.539 across ten degree-preserving controls, and outgoing distinct-use degree yields a Gini coefficient of 0.731. Across the 16 fixed sources, modularity ranges from 0.622 to 0.629, with a mean of 0.625. These measurements characterise concentrated reuse and modular structure without promoting a single image to a mean result.

The new outgoing-degree profiles also differ from the earlier 2,708-vertex core. None of the 16 fixed-repertoire sources rejects the pure discrete power-law family at five percent; bootstrap p-values range from 0.24190 to 0.62575. The representative fits an exponent of 2.044719 from degree one, with 524 tail observations and $p = 0.54640$. Its

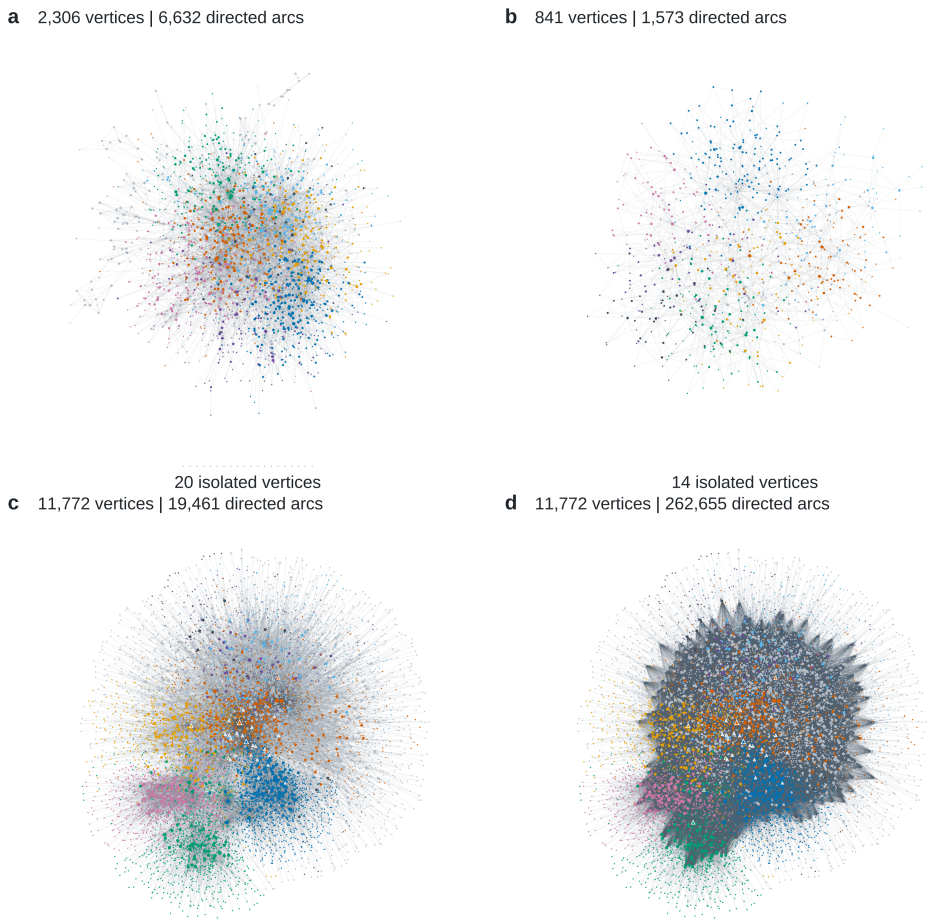


Fig. 2 Native proof networks on one explicit basis. a shows the compatible named Metamath fragment; b shows the first-source cumulative skeleton with fixed structural rules, including all certificate support; c shows first proofs in the complete height-three operator slice; d shows all enumerated inferences on those same vertices. Every vertex and unique undirected edge enters the drawing; only arrowheads disappear. Colours distinguish the eight largest communities in a, b, and c, with smaller communities in grey. d inherits c's coordinates, colours, and vertex areas; its tabulated modularity follows its own community search. Triangles mark the 29 basis roots. Areas increase with log reuse. Separate layouts do not measure comparable semantic distances

other 317 vertices carry zero outgoing degree. On this same tail, the cutoff power law and rounded lognormal yield descriptive log likelihoods of -815.753 and -816.024, versus -818.182 for the pure power law. Compatibility with the pure family therefore does not select it uniquely or establish an infinite scale-free growth mechanism (Table 4).

Table 4 Network metrics for Metamath, the cumulative skeleton, and two complete height-three projections

Metric	a Metamath	b Cumulative skeleton	c First proofs	d All inferences
Vertices N	2306	841	11772	11772
Directed arcs E	6632	1573	19461	262655
Largest-component vertices	2286	827	11772	11772
Isolated vertices	20	14	0	0
Mean local clustering C	0.0689	0.0202	0.0347	0.3382
Mean shortest path L	4.137	4.177	4.334	3.213
Diameter D	11	8	6	4
Modularity Q	0.568	0.622	0.675	0.058
Reference modularity Q_0	0.401	0.539	0.599	0.053
Small-world ratio σ_{DP}	3.990	0.636	0.592	0.417
Tail observations	205	524	63	41
Pure power law p	0.09895	0.54640	0.01080	0.36490

Table columns follow the figure panels. C, L, D, and Q refer to simple undirected projections. C assigns zero below degree two; L and D refer to the largest connected component and use every observed breadth-first-search (BFS) source. Panel a contains 20 isolates and panel b contains 14 unused basis roots; panels c and d remain connected. Q0 averages ten simple degree-preserving controls, with three Louvain starts for every observed and control graph. Panel b's control paths also use every source; the retained c and d controls estimate paths from 64 sources. The tail test for outgoing distinct-use degree fixes 20,000 bootstrap draws and refits both threshold and exponent. Appendix H reports all 32 new profiles and their scope.

The c–d pair isolates an edge-recording effect on identical statements: alternative edges raise clustering from 0.0347 to 0.3382, shorten the mean path from 4.334 to 3.213, and lower modularity from 0.675 to 0.058. No logical or vertex change explains that contrast. The ordinary proof-network profile therefore depends materially on how the representation records alternative derivations.

Newman [30], in *The Structure and Function of Complex Networks*, treats degree heterogeneity, clustering, paths, and correlations as complementary features. Humphries and Gurney [31], in *Network 'Small-World-Ness': A Quantitative Method for Determining Canonical Network Equivalence*, formalise the clustering–path comparison behind the small-world ratio. Under the declared degree-preserving reference, Metamath reaches a ratio of 3.990, whereas the skeleton reaches 0.636. Thus, the skeleton exhibits concentrated reuse, modular organisation, and a compatible conditional power-law tail, but its short paths do not establish small-world structure. These objects display distinct complex-network profiles rather than a division between networks and 'mere graphs'. A conditional tail fit does not establish infinite scale invariance, and network metrics complement rather than replace future-use tests [32, 33]. Ten null graphs support descriptive contrasts rather than a rank-test claim below five percent.

8 Discussion and scope

The experiments establish that proof validity, compact proof code, and productive premise use answer different questions. The architecture preserves every accepted certificate while the policy changes which proven formulae can take part in new inferences. Internal controls measure the computational consequences of that change. Fragment reuse alone does not determine the return from an active slot, as the utility problem for acquired knowledge predicts [19].

The coding library stores exact closed first proofs rather than shortest proofs. The structural extension separately derives general closed schemata by discharging open MP boundaries. The archive already shares identical subproofs in a directed acyclic graph (DAG), so shorter expanded terms do not demonstrate further compression of that graph. Repeated fragment use and conservative lemma synthesis preserve formal meaning; the costed future-growth test measures their operational value.

Human references expose differences that raw formula counts conceal. The locked comparison evaluates a previously unscored 963-target POI subset only after every search trajectory freezes; a separate independent audit then checks the outcome. These corpora neither supply a complete modal reference nor assess the full mathematical value of the generated library.

The native network study adds a structural result to the operational evidence. A proof-closed skeleton exhibits modular organisation and concentrated reuse, whereas the fixed-vertex reference shows that alternative proof edges can strongly alter the same network metrics. Verified certificates, human-statement coverage, and measured continuation complement this structural profile. The native network study and the earlier pair-selection trajectories retain their separate scopes.

The combinatorial claims also require distinct scopes. The exponential lower bound counts explicit theorem instances, whereas the result for canonical schemata establishes only unboundedness. Neither the theorems nor finite runs establish non-computability of a bounded slice, exponential growth of schemata, or the limiting topology of an infinite proof space. Since the two logics change their language, axioms, and rules together, they do not isolate logic alone as a cause.

The native comparison supplies a reproducible operating configuration with fixed structural rules. Goal-directed proof search and control towards prescribed graph properties define the next two research tracks. For knowledge-based AI, our method preserves certified knowledge, changes which statements inference may use, and accounts for the required resources.

9 Conclusions

We formulate the direct, inverse, and control cognition problems for formal knowledge. The direct cognition problem generates a proof skeleton without a final target theorem.

We develop a cumulative method that generates a semantic skeleton through checkable anchor statements and expandable proofs. The method addresses the direct cognition problem by separating archived proofs, active premises, and reusable lemma entries. Premise filtering, deferred replay, and reversible deactivation regulate immediate work; positive-marginal coding retains recurrent proof fragments; a structural compiler

derives general lemma schemata with elementary certificates. The theoretical results show that the method recovers proofs without strengthening the logic and that fixed capacity cannot accommodate a complete explicit representation as the size bound grows.

The experiments assess future capability rather than count retained formulae alone. In the 144-source continuation panel, learned pair selection with deferred replay raises mean novelty over ungated growth by 95.8% and 30.3% in the two propositional regimes, with eight positive pairs in each and a lower mean credited CPU cost for the chain. Modal outcomes and resource guards delimit the claim. Empirical Pareto sets retain the finite trade-offs among CPU, package bytes, and future output rather than name a universal winner.

The native architecture carries the accumulated method onto an explicit Metamath basis with checked structural rules and controlled access to certificate support. The locked 963-target test completes 160 continuations across 16 source blocks and passes independent replay. The registered comparison does not confirm an adaptive advantage; fixed structural rules supply the final repertoire. The locked comparison separates initial coverage from newly reached reference statements.

The new network comparison characterises the resulting certified skeleton alongside human mathematics and finite exhaustive growth. The first-source fixed skeleton contains 841 certificate vertices and reaches modularity 0.622; all 16 profiles quantify variation. Their outgoing-degree diagnostics do not reject a pure power-law tail, although alternative models also fit and the finite experiment establishes no limiting law. On the identical 11,772 vertices of the complete height-three slice, alternative proof edges raise clustering from 0.0347 to 0.3382 while lowering modularity from 0.675 to 0.058. These measurements connect useful finite proof libraries to network structure.

The principal result answers the direct question through an executable proof-preserving method, a future-use criterion, and an explicit cost account. It supports continued inference from finite certified skeletons in the tested regimes and connects their operational use to measurable network structure.

10 Reproducibility

The reproducibility records link every new numerical claim to its protocol, code, and source results. All 16 blind receipts, the one-shot evaluation, the completed pipeline, and the independent review refer to the corrected seal in Appendix G. The auditor rechecks permitted inference, target-instance certificates, common residuals, and the statistical decision.

Statements and Declarations

Data availability

We publicly provide source code for the main native workflow, self-contained tests, and compact checked results at https://github.com/nnevdokimov/certified-proof-skeleton. Frozen packages and the editorial manifest identify the protocols, code versions, input hashes, and numerical sources. We do not yet provide the complete experimental

record in a permanent archive. The source release does not include a reuse licence; third-party inputs retain their original licence requirements.

Conflicts of interest

The authors declare no conflict of interest concerning this article.

Funding

The research leading to these results received funding from the Basic Research Program at HSE University [HSE-BR-2025-025].

Acknowledgments

We used the HSE University supercomputer to conduct the computational research.

References

[1] Metamath Project. (2026). *set.mm: ZFC set theory database, fixed experimental version*. https://github.com/metamath/set.mm/commit/584b6852f57f281e9be1793332f1e52e077693c8

[2] Bachmair, L., & Ganzinger, H. (2001). Resolution theorem proving. In J. A. Robinson & A. Voronkov (Eds.), *Handbook of automated reasoning* (pp. 19–99). Elsevier. https://doi.org/10.1016/B978-044450813-3/50004-7

[3] Wernhard, C. (2026). Generating theorems by generating proof structures. *Automated Reasoning: IJCAR 2026, Lecture Notes in Computer Science*, *16688*, 41–61. https://doi.org/10.1007/978-3-032-32589-1_3

[4] Schulz, S. (2002). E – a brainiac theorem prover. *AI Communications*, *15*(2–3), 111–126. https://doi.org/10.3233/EAI-2002-260

[5] Kovács, L., & Voronkov, A. (2013). First-order theorem proving and Vampire. *Computer Aided Verification*, 1–35. https://doi.org/10.1007/978-3-642-39799-8_1

[6] Riazanov, A., & Voronkov, A. (2003). Limited resource strategy in resolution theorem proving. *Journal of Symbolic Computation*, *36*(1–2), 101–115. https://doi.org/10.1016/S0747-7171(03)00040-3

[7] Hoder, K., & Voronkov, A. (2011). Sine qua non for large theory reasoning. *Automated Deduction – CADE-23*, 299–314. https://doi.org/10.1007/978-3-642-22438-6_23

[8] Blanchette, J. C., Greenaway, D., Kaliszyk, C., Kühlwein, D., & Urban, J. (2016). A learning-based fact selector for Isabelle/HOL. *Journal of Automated Reasoning*, *57*(3), 219–244. https://doi.org/10.1007/s10817-016-9362-8

[9] Jakubův, J., & Urban, J. (2017). ENIGMA: Efficient learning-based inference guiding machine. *Intelligent Computer Mathematics*, 292–302. https://doi.org/10.1007/978-3-319-62075-6_20

[10] Colton, S. (2002). *Automated theory formation in pure mathematics*. Springer. https://doi.org/10.1007/978-1-4471-0147-5

[11] Smallbone, N., Johansson, M., Claessen, K., & Algehed, M. (2017). Quick specifications for the busy programmer. *Journal of Functional Programming*, *27*, e18. https://doi.org/10.1017/S0956796817000090

[12] Claessen, K., Johansson, M., Rosén, D., & Smallbone, N. (2013). Automating inductive proofs using theory exploration. *Automated Deduction – CADE-24, Lecture Notes in Computer Science*, *7898*, 392–406. https://doi.org/10.1007/978-3-642-38574-2_27

[13] Johansson, M., Rosén, D., Smallbone, N., & Claessen, K. (2014). Hipster: Integrating theory exploration in a proof assistant. *Intelligent Computer Mathematics*, 108–122. https://doi.org/10.1007/978-3-319-08434-3_9

[14] Singher, E., & Itzhaky, S. (2021). Theory exploration powered by deductive synthesis. *Computer Aided Verification, Lecture Notes in Computer Science*, *12760*, 125–148. https://doi.org/10.1007/978-3-030-81688-9_6

[15] McCasland, R. L., Bundy, A., & Smith, P. F. (2017). MATHsAiD: Automated mathematical theory exploration. *Applied Intelligence, 47*, 585–606. https://doi.org/10.1007/s10489-017-0954-8

[16] Wernhard, C., & Bibel, W. (2024). Investigations into proof structures. *Journal of Automated Reasoning*, *68*, 24. https://doi.org/10.1007/s10817-024-09711-8

[17] Rawson, M., Wernhard, C., Zombori, Z., & Bibel, W. (2023). Lemmas: Generation, selection, application. *Automated Reasoning with Analytic Tableaux and Related Methods*, 153–174. https://doi.org/10.1007/978-3-031-43513-3_9

[18] Wernhard, C., & Zombori, Z. (2025). Mathematical knowledge bases as grammar-compressed proof terms: Exploring Metamath proof structures. *arXiv Preprint arXiv:2505.12305*. https://doi.org/10.48550/arXiv.2505.12305

[19] Minton, S. (1990). Quantitative results concerning the utility of explanation-based learning. *Artificial Intelligence*, *42*(2–3), 363–391. https://doi.org/10.1016/0004-3702(90)90059-9

[20] Kaliszyk, C., & Urban, J. (2015). Learning-assisted theorem proving with millions of lemmas. *Journal of Symbolic Computation*, *69*, 109–128. https://doi.org/10.1016/j.jsc.2014.09.032

[21] Wang, H., Xin, H., Zheng, C., Liu, Z., Cao, Q., Huang, Y., Xiong, J., Shi, H., Xie, E., Yin, J., Li, Z., & Liang, X. (2024). LEGO-Prover: Neural theorem proving with growing libraries. *The Twelfth International Conference on Learning*

Representations. https://proceedings.iclr.cc/paper_files/paper/2024/hash/85dca46374dc0f27b4bb5f265b3d17f0-Abstract-Conference.html

[22] Yang, K., Swope, A., Gu, A., Chalamala, R., Song, P., Yu, S., Godil, S., Prenger, R., & Anandkumar, A. (2023). LeanDojo: Theorem proving with retrieval-augmented language models. *Advances in Neural Information Processing Systems: Datasets and Benchmarks*. https://doi.org/10.52202/075280-0944

[23] Poesia, G., Broman, D., Haber, N., & Goodman, N. D. (2024). Learning formal mathematics from intrinsic motivation. *Advances in Neural Information Processing Systems*, *37*. https://doi.org/10.52202/079017-1362

[24] Ota, K., Osa, T., & Harada, T. (2026). Self-supervised theorem discovery in a formal axiomatic system. *arXiv Preprint arXiv:2606.28747*. https://doi.org/10.48550/arXiv.2606.28747

[25] Lin, X., Cao, Q., Huang, Y., Yang, Z., Liu, Z., Li, Z., & Liang, X. (2024). ATG: Benchmarking automated theorem generation for generative language models. *Findings of the Association for Computational Linguistics: NAACL 2024*, 4465–4480. https://doi.org/10.18653/v1/2024.findings-naacl.279

[26] Tomaszuk, D., Szeremeta, Ł., & Korniłowicz, A. (2023). MMLKG: Knowledge graph for mathematical definitions, statements and proofs. *Scientific Data*, *10*, 791. https://doi.org/10.1038/s41597-023-02681-3

[27] Li, X., Peng, N., Severini, S., & Shafto, P. (2026). The network structure of mathlib. *arXiv Preprint arXiv:2604.24797*. https://doi.org/10.48550/arXiv.2604.24797

[28] Flajolet, P., & Sedgewick, R. (2009). *Analytic combinatorics*. Cambridge University Press. https://doi.org/10.1017/CBO9780511801655

[29] Cook, S. A., & Reckhow, R. A. (1979). The relative efficiency of propositional proof systems. *Journal of Symbolic Logic*, *44*(1), 36–50. https://doi.org/10.2307/2273702

[30] Newman, M. E. J. (2003). The structure and function of complex networks. *SIAM Review*, *45*(2), 167–256. https://doi.org/10.1137/S003614450342480

[31] Humphries, M. D., & Gurney, K. (2008). Network 'small-world-ness': A quantitative method for determining canonical network equivalence. *PLoS ONE*, *3*(4), e0002051. https://doi.org/10.1371/journal.pone.0002051

[32] Clauset, A., Shalizi, C. R., & Newman, M. E. J. (2009). Power-law distributions in empirical data. *SIAM Review*, *51*(4), 661–703. https://doi.org/10.1137/070710111

[33] Broido, A. D., & Clauset, A. (2019). Scale-free networks are rare. *Nature Communications*, *10*, 1017. https://doi.org/10.1038/s41467-019-08746-5